AES-128 Overview

AES-128 performs 10 rounds on a 4×4 byte state matrix. Each round (except the last) applies:

1. **SubBytes** - non-linear S-Box substitution,
2. **ShiftRows** - cyclic row shifts,
3. **MixColumns** - linear mixing in GF(2^8),
4. **AddRoundKey** - XOR with the round key.

Round 10 omits MixColumns. The key schedule expands the 128-bit master key K_0 into eleven 128-bit round keys $K_0 \ldots K_{10}$.

1 Question 1: Fault Round Identification

1.1 Method: ΔC Pattern Analysis

For each ciphertext pair (C, C') the XOR difference $\Delta C = C \oplus C'$ is computed and the number of non-zero bytes is counted.

Table 1: Fault diffusion fingerprint by injection round

Fault injected before ...	**Differing bytes in ΔC**	**Pattern**
Round 10 MixColumns (skipped)	1	1 byte affected
Round 9 SubBytes	4	1 diagonal
Round 8 SubBytes	16	all bytes

1.2 Observed Results

All 10 ciphertext pairs show **exactly 16 differing bytes** across all positions $\{0, 1, \ldots, 15\}$

Conclusion: The fault was injected before Round 8.

2 Question 2: Round Key Obtained Directly from DFA

The standard DFA attack on AES-128 with a Round-8 fault directly recovers $\mathbf{K_{10}}$ - the last round key.

Why $\mathbf{K_{10}}$? Working backwards from the output ciphertext, the attacker can reach the state *just before AddRoundKey of Round 10* by stripping off K_{10}. At that point the known fault-propagation structure (four diagonal groups of 4 bytes each) provides enough constraints to enumerate and verify candidate key bytes one byte at a time using the inverse S-Box and GF(2^8) arithmetic.

Round key directly recovered: K_{10}

```
K10 (hex)  : BBF46A0A9909E151B91BFEC8DFE63610
K10 (bytes): [187, 244, 106, 10, 153, 9, 225, 81,
              185, 27, 254, 200, 223, 230, 54, 16]
```

Listing 1: Recovered K_{10}

3 Question 3: Number of Key Bytes Recovered

All 16 / 16 bytes of K_{10} were recovered.

The 16 key bytes are partitioned into four *diagonal groups* of 4 bytes each, corresponding to the four diagonals of the AES state matrix after ShiftRows:

4 Question 4: Python Script - `dfa_8.py`

The complete offline-phase script is submitted as `dfa_8.py`. Below is a description of each functional component.

4.1 Reading Ciphertexts (Step a)

The plaintext, correct ciphertext, and ten faulty ciphertexts are taken as Python `bytes` literals inside `main()`. This mirrors the data collected during the online phase.

4.2 Automatic Fault-Round Detection (Step b)

The helper `detect_fault_round(correct_ct, faulty_ct)` counts the number of differing bytes via XOR. It returns 9 when 4 bytes differ and **8** when all 16 bytes differ. All ten pairs return 16 differing bytes, so the fault round is automatically identified as **Round 8**.

4.3 K_{10} Recovery via DFA (Step c)

Convert Round-8 faults to Round-9 equivalent faults. A Round-8 fault produces 16 differing output bytes. The function `convert_r8faults_bytes` decomposes each faulty ciphertext into four synthetic "Round-9 faults", one per diagonal group, by replacing only the relevant 4 positions with the faulty bytes while keeping the rest from the correct ciphertext. Each original pair thus yields four Round-9 fault pairs.

Byte-wise exhaustive search. `crack_bytes` iterates over each diagonal group independently. For each of the 4 positions in the group it tries all 256 candidate key bytes k and checks the consistency condition:

$$\text{InvSBox}\big(C[i] \oplus k\big) \;\oplus\; \text{InvSBox}\big(C'[i] \oplus k\big) \;=\; \delta_i$$

where δ_i is the expected fault difference at position i derived from $\text{GF}(2^8)$ MixColumns coefficients. Only the candidate k consistent with *all* fault pairs survives - this is typically a unique solution after 2 pairs.

4.4 Reverse Key Schedule to Obtain K_0 (Step d)

`reverse_key_schedule(k10)` inverts the AES-128 key expansion. Starting from K_{10} it reconstructs words W_{39} down to W_0 using the inverse relation:

$$W_{i-4} = W_i \oplus W_{i-1} \quad (i \bmod 4 \neq 0),$$
$$W_{i-4} = W_i \oplus \mathrm{SubWord}(\mathrm{RotWord}(W_{i-1})) \oplus \mathrm{Rcon}[i/4 - 1] \quad (i \bmod 4 = 0)$$

4.5 Verification (Step e)

The recovered K_0 is used to encrypt the known plaintext with AES-ECB and the result is compared byte-for-byte with the correct ciphertext. The script prints **PASS** on success.

4.6 Output to `roundkeys.txt` (Step f)

All eleven round keys (K_0 through K_{10}) are written to `roundkeys.txt` in the required format.

```
round 0 :
   [108,136,219,37,132,227,185,26,69,248,19,186,148,184,162,35]
round 1 : [1,178,253,7,133,81,68,29,192,169,87,167,84,17,245,132]
...
round 10:
   [187,244,106,10,153,9,225,81,185,27,254,200,223,230,54,16]
```

Listing 2: Sample output - roundkeys.txt

5 Question 5: Detailed Report

5.1 Step-by-Step Analysis

Step 1. Fault round detection. Compute $\Delta C = C \oplus C'$ for all pairs. Count non-zero bytes. 16 non-zero bytes across all pairs $\Rightarrow$ fault at Round 8 (before MixColumns).

Step 2. Synthetic Round-9 fault generation. Each Round-8 faulty ciphertext is split into four synthetic Round-9 faults (one per AES diagonal). Ten original pairs produce 40 Round-9 fault pairs.

Step 3. Diagonal-group DFA. For each of the four diagonal groups, try all 256 byte candidates for each of the 4 positions. A candidate is valid if it satisfies the inverse MixColumns consistency check across all available fault pairs. After processing just **2 original fault pairs** (= 8 Round-9 pairs) every byte is uniquely determined.

Step 4. K_{10} assembly. The 16 individually recovered bytes are concatenated to form the 128-bit last round key `BBF46A0A 9909E151 B91BFEC8 DFE63610`.

Step 5. Key-schedule reversal. Applying the inverse key-expansion recursion ten times to K_{10} yields K_0 = `6C88DB25 84E3B91A 45F813BA 94B8A223`.

Step 6. Verification. AES-ECB encryption of the known plaintext under K_0 reproduces the correct ciphertext exactly - verification **PASS**.

5.2 Time Taken to Obtain the Round Key

Phase	Time
DFA analysis (K_{10} recovery)	**0.028 seconds**
Key-schedule reversal	< 0.001 seconds
Total	**$\approx$ 0.03 seconds**

The attack is essentially instantaneous. The bottleneck is the $256 \times 4 \times 4 = 4096$ candidate evaluations per fault pair, each requiring only table lookups - trivially fast on modern hardware.

5.3 Minimum Ciphertext Pairs Required

Table 2: Minimum fault pairs needed per K_{10} byte

Byte	K10 (hex)	K10 (dec)	Min pairs	Diagonal group
0	`0xBB`	187	2	0
1	`0xF4`	244	2	1
2	`0x6A`	106	2	2
3	`0x0A`	10	2	3
4	`0x99`	153	2	1
5	`0x09`	9	2	2
6	`0xE1`	225	2	3
7	`0x51`	81	2	0
8	`0xB9`	185	2	2
9	`0x1B`	27	2	3
10	`0xFE`	254	2	0
11	`0xC8`	200	2	1
12	`0xDF`	223	2	3
13	`0xE6`	230	2	0
14	`0x36`	54	2	1
15	`0x10`	16	2	2

Minimum fault pairs for complete K_{10} recovery: 2.

Each original fault pair is converted into 4 Round-9 synthetic faults (one per diagonal group), so 2 original pairs suffice to uniquely determine all 16 key bytes. In the worst case, a single fault pair may leave several candidate ambiguities per byte; a second pair resolves them.

5.4 References

During the development of this assignment, the following external resources were consulted:

Conceptual Understanding:
Quarkslab Blog - *Differential Fault Analysis on White-Box AES Implementations*

Implementation Reference:
SideChannelMarvels / JeanGrey - *phoenixAES* (open-source DFA library)
The implementation ideas for the core DFA solver - in particular the fault-map structure, the diagonal-group decomposition of Round-8 faults into synthetic Round-9 faults, and the candidate-filtering loop - were drawn from this reference implementation and adapted for this assignment.

6 Summary of Results

Table 3: Complete DFA attack results

Parameter	Value
Fault model	Single-byte fault
Fault round identified	Round 8 (before MixColumns)
Round key recovered	K_{10} (directly)
Bytes recovered	16 / 16
K_{10}	`BBF46A0A9909E151B91BFEC8DFE63610`
K_0	`6C88DB2584E3B91A45F813BA94B8A223`
Verification	**PASS**
DFA analysis time	0.0124 s
Minimum fault pairs	**2**

All Round Keys

```
round  0 : [108,136,219, 37,132,227,185, 26, 69,248,
   19,186,148,184,162, 35]  <- K0
round  1 : [  1,178,253,  7,133, 81, 68, 29,192,169, 87,167, 84,
   17,245,132]
round  2 : [129, 84,162, 39,  4,  5,230,
   58,196,172,177,157,144,189, 68, 25]
round  3 : [255, 79,118, 71,251, 74,144,125, 63,230, 33,224,175,
   91,101,249]
round  4 : [206,  2,239, 62, 53, 72,127, 67, 10,174,
   94,163,165,245, 59, 90]
round  5 : [ 56,224, 81, 56, 13,168, 46,123,  7,
   6,112,216,162,243, 75,130]
round  6 : [ 21, 83, 66,  2, 24,251,108,121, 31,253, 28,161,189,
   14, 87, 35]
round  7 : [254,  8,100,120,230,243,  8,  1,249, 14, 20,160, 68,
    0, 67,131]
round  8 : [ 29, 18,136, 99,251,225,128, 98,  2,239,148,194,
   70,239,215, 65]
round  9 : [217, 28, 11, 57, 34,253,139, 91, 32, 18,
   31,153,102,253,200,216]
round 10 : [187,244,106, 10,153,  9,225, 81,185,
   27,254,200,223,230, 54, 16]  <- K10
```

Listing 3: roundkeys.txt - all 11 round keys